DEVELOPER REFERENCE

Generated 12 October 2026 · package version v1.7.1

Using `peoplelogy/laravel-xero-bridge` — Invoices

A practical reference for the invoice flows most applications need: creating a draft, getting an invoice to PAID, sending it with the buyer's TIN, BRN and CC list, finding one, and listing them — what to pass, what goes over the wire, what comes back, and what goes wrong.

This is one of the usage guides in this folder. **Xero usage** covers how to reach the package (the facade, connections, return shapes, events, the full exception list); **MyInvois usage** covers validating a Malaysian TIN with LHDN. This one assumes the facade is imported:

```
use Peoplelogy\XeroBridge\Facades\XeroBridge;
```

The package contains no bookkeeping logic. *When* to raise an invoice and *what* goes on it stay yours to decide. Every flow below can be run by hand from the test console at `/xero/console` (off until `XERO_CONSOLE_ENABLED=true`; writes only into a Demo Company or an allow-listed organisation).

> **Version note.** Written against `peoplelogy/laravel-xero-bridge` v1.7.1. Method signatures are read from the package source; request and response samples are reproduced from `02-invoices` and `06-recipes` in the folder above this one, which are the authority wherever the two disagree. The GUIDs and figures in them are illustrative.

Contents

1. Flow 1 — create a DRAFT invoice
2. Flow 2 — create an invoice that ends up PAID
3. Flow 3 — TIN, BRN, CC list, then send
4. Flow 4 — find one invoice
5. Flow 5 — list invoices
6. Errors you will meet on invoices
7. Things that differ per organisation

Flow 1 — create a DRAFT invoice

```
$invoice = XeroBridge::invoices()->create([
    'Type' => 'ACCREC',                                    // REQUIRED
    'Status' => 'DRAFT',
    'Contact' => ['ContactID' => '1111...'],               // ContactID ALONE
    'Date' => '2026-03-02',
    'DueDate' => '2026-04-01',
    'Reference' => 'CCF-2026-0042',
    'LineItems' => [[
        'Description' => 'Leadership training, 2 days',
        'Quantity' => 1,
        'UnitAmount' => 4800,
        'AccountCode' => '200',      // omit to use the connection default
        'TaxType' => 'OUTPUT',       // omit to use the connection default
    ]],
]);
```

Parameters

Field	Required	Notes
`Type`	**yes**	`ACCREC` (sales) or `ACCPAY` (bill). Missing throws `InvalidInvoicePayloadException` before anything is sent.
`Contact`	**yes**	`['ContactID' => ...]` — see the gotcha below.
`LineItems`	**yes**	At least one. `Description` is the minimum.
`Status`	no	Omitted creates a `DRAFT`.
`Date`, `DueDate`, `Reference`	no	Plain `Y-m-d`.
`AccountCode`, `TaxType`, `CurrencyCode`, `BrandingThemeID`	no	Filled from the connection defaults **only where you left them out**.

On the wire

`POST /api.xro/2.0/Invoices`, with an `Idempotency-Key` header:

```
{
  "Invoices": [
    {
      "Type": "ACCREC",
      "Contact": { "ContactID": "11111111-1111-1111-1111-111111111111" },
      "Date": "2026-03-02",
      "DueDate": "2026-04-01",
      "Reference": "CCF-2026-0042",
      "Status": "DRAFT",
      "LineItems": [
        {
          "Description": "Leadership training, 2 days",
          "Quantity": 1,
          "UnitAmount": 4800,
          "TaxType": "OUTPUT",
          "AccountCode": "200"
        }
      ],
      "CurrencyCode": "MYR"
    }
  ]
}
```

Response

The method returns the inner invoice, not the envelope:

```
{
  "Type": "ACCREC",
  "InvoiceID": "22222222-2222-2222-2222-222222222222",
  "InvoiceNumber": "INV-0042",
  "Reference": "CCF-2026-0042",
  "Status": "DRAFT",
  "AmountDue": 5184.00,
  "AmountPaid": 0.00,
  "SubTotal": 4800.00,
  "TotalTax": 384.00,
  "Total": 5184.00,
  "CurrencyCode": "MYR",
  "DateString": "2026-03-02T00:00:00",
  "DueDateString": "2026-04-01T00:00:00",
  "SentToContact": false,
  "Contact": { "ContactID": "1111...", "Name": "Acme Sdn Bhd" },
  "LineItems": [
    {
      "LineItemID": "33333333-3333-3333-3333-333333333333",
      "Description": "Leadership training, 2 days",
      "UnitAmount": 4800.00,
      "TaxType": "OUTPUT",
      "TaxAmount": 384.00,
      "LineAmount": 4800.00,
      "AccountCode": "200",
      "Quantity": 1.0000
    }
  ]
}
```


Persist `InvoiceID` and `InvoiceNumber`. Note that **tax is computed server-side** — you sent 4800, Xero returned a `Total` of 5184.

Gotchas

- **`Contact` must carry `ContactID` and nothing else.** With `ContactID` plus any other field, Xero applies those fields to the *contact record* and **deletes any `ContactPersons` you left out**. The package refuses this with `UnsafeContactPayloadException` rather than letting it happen. If you genuinely mean it, chain the opt-in in front of the call — `XeroBridge::invoices()->withContactMutation()->create([...])`. It returns a copy, so on a line of its own it changes nothing and the payload is still refused. It is irreversible and nobody notices for weeks.
- A `DRAFT` can be deleted outright. Once `AUTHORISED` it can only be voided, and it stays visible in Xero forever.
- An unknown `AccountCode` is a 400 → `XeroValidationException`, with Xero's own message in `validationErrors()`.

- The `Idempotency-Key` is retained by Xero for **six minutes**. It protects an immediate network retry; it does **not** deduplicate a queued job retried an hour later. Persist `InvoiceID` for that — or name the record the invoice is for, `XeroBridge::invoices()->for($order)->create([...])`, with the write ledger switched on (`XERO_WRITES_LEDGER=true` and its table migrated), and a second attempt for that record throws `XeroWriteAlreadyClaimedException` instead of reaching Xero.

Flow 2 — create an invoice that ends up PAID

> **PAID is not a status you can set**
>
> Xero rejects `"Status": "PAID"` on create. `PAID` is what Xero moves an invoice to once payments cover it in full. There is no single call for this.

Three steps, in this order:

```
// 1. AUTHORISED, not DRAFT — a payment cannot attach to a draft
$invoice = XeroBridge::invoices()->create([
    'Type' => 'ACCREC',
    'Status' => 'AUTHORISED',
    'Contact' => ['ContactID' => $contactId],
    'LineItems' => [[ 'Description' => '...', 'Quantity' => 1, 'UnitAmount' => 4800 ]],
]);

// 2. pay Xero's own AmountDue, never the figure you sent
$payment = XeroBridge::payments()->createForInvoice(
    $invoice['InvoiceID'],
    (float) $invoice['AmountDue'],   // 5184.00, not 4800
    '090',                           // a BANK / payments-enabled account code
    '2026-03-02',                    // optional, defaults to today
);

// 3. read it back — do not assume
$settled = XeroBridge::invoices()->find($invoice['InvoiceID']);
// $settled['Status'] === 'PAID'
```

`createForInvoice()` parameters

Argument	Type	Notes
`$invoiceId`	`string`	Must be **AUTHORISED**.
`$amount`	`float|int|string`	Use the invoice's `AmountDue`.
`$accountCode`	`?string`	Type `BANK`, or with "enable payments to this account" on. Discover with `settings()->paymentAccounts()`. Throws `InvalidArgumentException` if null and no connection default.
`$date`	`DateTimeInterface|string|null`	Defaults to today.
`$extra`	`array`	Merged into the payload.

On the wire

```
POST /api.xro/2.0/Payments
{
  "Payments": [
    {
      "Invoice": { "InvoiceID": "c7c37b83-ac95-45ea-88ba-8ad83a5f22fe" },
      "Account": { "Code": "090" },
      "Amount": 5184,
      "Date": "2026-03-02"
    }
  ]
}
```

Gotchas

- **Pay `AmountDue`, not your own total.** Tax and rounding are applied server-side. Paying the pre-tax figure leaves the invoice `AUTHORISED` with a balance, not `PAID`.
- **A `PAID` invoice can never be changed.** `InvoiceTransitions` lists no legal transition out of it. To correct one, reverse the payment first with `payments()->delete($paymentId)`, then void.
- Overpaying is rejected by Xero — check `AmountDue` before applying.
- A successful 200 can still carry a `Warnings` array (a `CurrencyRate` warning, typically). The package logs it; check it yourself if a wrong foreign-currency amount would matter.

Flow 3 — TIN, BRN, CC list, then send

Three facts shape this flow, and all three surprise people.

> **`POST /Invoices/{id}/Email` takes an empty body**
>
> No `to`, no `cc`, no `bcc`, no `subject`, no `body`, no `replyTo`. Xero's own OpenAPI spec types the request body as `RequestEmpty`. The package sends `[]` on the wire and has a test asserting it stays empty.

1. **TIN and BRN live on the contact, not the invoice.** In a Malaysian organisation the TIN goes in `TaxNumber` (the field is relabelled per region — ABN in Australia, VAT Number in the UK); the BRN is `CompanyNumber`. Both max 50 characters.
2. **`ContactPersons` with `IncludeInEmails` is the only CC mechanism.** Maximum five. The list you send **replaces** the stored one wholesale, so build it in full every time. It is **per contact, not per send** — everyone on it is copied on every future invoice for that customer until you change it back.
3. **Xero only emails a SUBMITTED, AUTHORISED or PAID invoice**, so a DRAFT must be approved first.

Validate the TIN before you store it

Check the buyer's TIN and BRN with LHDN Malaysia before they are written onto the contact, rather than trusting whatever was typed:

```
use Peoplelogy\XeroBridge\MyInvois\Facades\MyInvois;
use Peoplelogy\XeroBridge\MyInvois\IdType;

if (! MyInvois::for($customer)->validate($tin, IdType::BRN, $brn)) {
    // HASiL has no record of this pair. Ask the buyer to confirm both values
    // rather than writing them to the contact and on to the invoice.
}
```

It needs **both** values (LHDN validates them as a pair since 1 August 2026), a negative answer is `false` rather than an exception, and a match proves only that the pair exists in HASiL's records. Turning it on, its errors and the optional records of each check are in the **MyInvois usage** guide.

Then update the contact, authorise, send

```
// (a) the buyer's details, onto the CONTACT
XeroBridge::contacts()->update($contactId, [
    'CompanyNumber' => '202401012345',   // BRN
    'TaxNumber' => 'C12345678901',       // TIN
    'ContactPersons' => [
        ['FirstName' => 'Siti', 'LastName' => 'Rahman',
         'EmailAddress' => 'ap@acme.com', 'IncludeInEmails' => true],
        ['FirstName' => 'Finance', 'LastName' => 'Team',
         'EmailAddress' => 'finance@acme.com', 'IncludeInEmails' => true],
    ],
]);

// (b) Xero refuses to email a DRAFT
XeroBridge::invoices()->authorise($invoiceId);

// (c) send. 204 No Content, so this returns void.
XeroBridge::invoices()->email($invoiceId);
```

On the wire

`POST /Contacts/{ContactID}`:

```
{
  "Contacts": [
    {
      "CompanyNumber": "202401012345",
      "TaxNumber": "C12345678901",
      "ContactPersons": [
        { "FirstName": "Siti", "LastName": "Rahman",
          "EmailAddress": "ap@acme.com", "IncludeInEmails": true }
      ]
    }
  ]
}
```

then `POST /Invoices/{InvoiceID}` with nothing but a status:

```
{ "Invoices": [ { "Status": "AUTHORISED" } ] }
```

then `POST /Invoices/{InvoiceID}/Email` with an empty body.

Who actually receives it

The contact's own `EmailAddress`, plus every `ContactPerson` whose `IncludeInEmails` is `true`. That is the entire mechanism.

Gotchas

- **You cannot copy someone on one invoice only.** Changing who is copied means changing the contact, and that applies to every subsequent invoice for that customer.
- **The template and sender are not yours.** Subject and body come from the organisation's email template, configured in Xero's UI. The sender is the user who authorised the app connection.
- **You cannot read back what was sent.** There is no API for the rendered email. If you need an audit trail, log the `ContactPersons` you set and the moment you called `email()`.
- If none of that fits, send it yourself: `invoices()->pdf($id)` for the bytes or `invoices()->onlineUrl($id)` for the public link, build your own Mailable, then call `invoices()->markAsSent($id)` so Xero's UI still shows it as sent.

Flow 4 — find one invoice

```
$invoice = XeroBridge::invoices()->find('INV-0042');   // or the InvoiceID GUID

if ($invoice === null) {
    // no such invoice — this is NOT an exception
}
```

Argument	Notes
`$idOrNumber`	An `InvoiceID` GUID **or** a human `InvoiceNumber` such as `INV-0042`.

`GET /api.xro/2.0/Invoices/{id}`. Returns the same shape as flow 1, including full `LineItems` and any `Payments` applied.

Gotcha: returns `null` for "not found" rather than throwing, so `??` and `is_null()` work naturally — but an unchecked `$invoice['Status']` will fatal on a miss.

Flow 5 — list invoices

```
use Peoplelogy\XeroBridge\Filters\InvoiceFilter;

$invoices = XeroBridge::invoices()->list(
    InvoiceFilter::make()
        ->statuses(['AUTHORISED', 'PAID'])
        ->type('ACCREC')
        ->dateBetween('2026-01-01', '2026-03-31')
        ->orderBy('Date', 'DESC')
        ->page(1, 25)
);
```

Returns `list<array>` — one page. For everything matching, `all()` returns a `LazyCollection` and walks Xero's pagination for you:

```
XeroBridge::invoices()->all($filter)->each(function (array $invoice) {
    // ...
});
```

Filter methods

Method	Notes
`statuses(string|array)`	`DRAFT`, `SUBMITTED`, `AUTHORISED`, `PAID`, `VOIDED`, `DELETED`
`type(string)`	`ACCREC` or `ACCPAY`
`contactIds()`, `contactName()`	
`invoiceNumbers()`, `ids()`, `reference()`	
`dateBetween($from, $to)`	Either side may be null
`modifiedSince($when)`	Sent as the `If-Modified-Since` header, not a query param
`orderBy($field, $dir)`	
`page($page, $pageSize)`, `pageSize()`	
`summaryOnly()`	Lighter rows, no line items
`createdByMyApp()`	Only invoices this app created
`whereRaw()`, `whereEquals()`, `whereGuid()`	Escape hatches

Gotchas

- **Always page.** An unbounded `/Invoices` on a live organisation is the quickest way to spend the daily rate limit. `all()` pages at 100 unless told otherwise.

- **Xero caps a page at 1000 and silently clamps anything larger**, so `pageSize()` throws outside 1–1000 rather than let a typo'd `5000` look like it worked.
- `list()` gives one page and ignores the rest. `all()` is the one that walks.
- `summaryOnly()` omits `LineItems` — don't reach into them on a summary row.
- `modifiedSince()` is a header, so it does not appear in the query string when debugging.

Errors you will meet on invoices

Every one extends `Peoplelogy\XeroBridge\Exceptions\XeroBridgeException`; the **Xero usage** guide has the full list and the accessors.

Exception	Cause	What to do
`InvalidInvoicePayloadException`	`Type` missing on create, or `LineItems` without a `LineItemID` on update	Nothing was sent. To replace every line on purpose, chain `replacingLineItems()` in front of `update()`
`UnsafeContactPayloadException`	An invoice `Contact` block carries `ContactID` **plus** other fields	Send `ContactID` alone — or, to update the contact on purpose, chain `withContactMutation()` in front of the call
`InvalidInvoiceTransitionException`	e.g. voiding a DRAFT	Checked locally against `InvoiceTransitions`, so the message names the current status
`InvoiceCannotBeVoidedException`	Voiding an `AUTHORISED` invoice that still has payments applied — the message names them	Remove each with `payments()->delete($paymentId)`, then void
`XeroValidationException`	400 from Xero: unknown `AccountCode`, missing contact, bad `TaxType`	Fix the payload. `validationErrors()` names the element.
`XeroWriteAlreadyClaimedException`	The write ledger already holds this invoice for that record (`for($order)`)	**Stop — do not retry.** `xeroId()` is the invoice already created
`XeroRateLimitException`	429	`$this->release($e->retryAfter() ?? 60)` in a queued job — never sleep inline

```php
use Peoplelogy\XeroBridge\Exceptions\XeroBridgeException;
use Peoplelogy\XeroBridge\Exceptions\XeroValidationException;

try {
    $invoice = XeroBridge::invoices()->for($order)->create($payload);
} catch (XeroValidationException $e) {
    report($e);                      // $e->validationErrors() holds Xero's own messages
} catch (XeroBridgeException $e) {
    report($e);
}
```

Things that differ per organisation

Nothing below should ever be hardcoded. These differ between Xero organisations, so a value that works in the sandbox will be rejected in production.

What	How to discover it
Account codes	`XeroBridge::settings()->accounts()` — a `200` sales account in one org is `4000` in another
Payment accounts	`XeroBridge::settings()->paymentAccounts()` — type `BANK` or payments-enabled only
Tax types	`XeroBridge::settings()->taxRates()` — the `TaxType` code, not the label
Branding themes	`XeroBridge::settings()->brandingThemes()`
Base currency, region, `IsDemoCompany`	`XeroBridge::settings()->organisation()`

This is also why three of the four per-connection defaults ship **no value at all**. Only `XERO_CURRENCY` has one, and only because a base currency is a property of the organisation rather than of a line:

Setting	Ships a default?
`XERO_ACCOUNT_CODE`	no — the `200` fallback was removed in v1.1.0
`XERO_TAX_TYPE`	no
`XERO_BRANDING_THEME_ID`	no
`XERO_CURRENCY`	yes, `MYR`

A wrong-but-present account code is worse than a missing one: Xero accepts the invoice and posts it to whatever account that code happens to be in that organisation, and nobody notices until Finance reconciles. With no default, Xero rejects the invoice and names the problem.

`XERO_TAX_TYPE` is unset for a second reason. Tax is frequently **per line**, not per invoice — under Malaysian SST, for instance, training is 8% while education and rental are 6%, so a single invoice can legitimately carry two rates. A connection-wide default would quietly stamp the wrong one on a venue recharge. Set it only if every line of every invoice on that connection genuinely carries the same rate.

None of these are cached by the package. The right cache key, TTL and invalidation depend entirely on the consuming application, so caching belongs there.